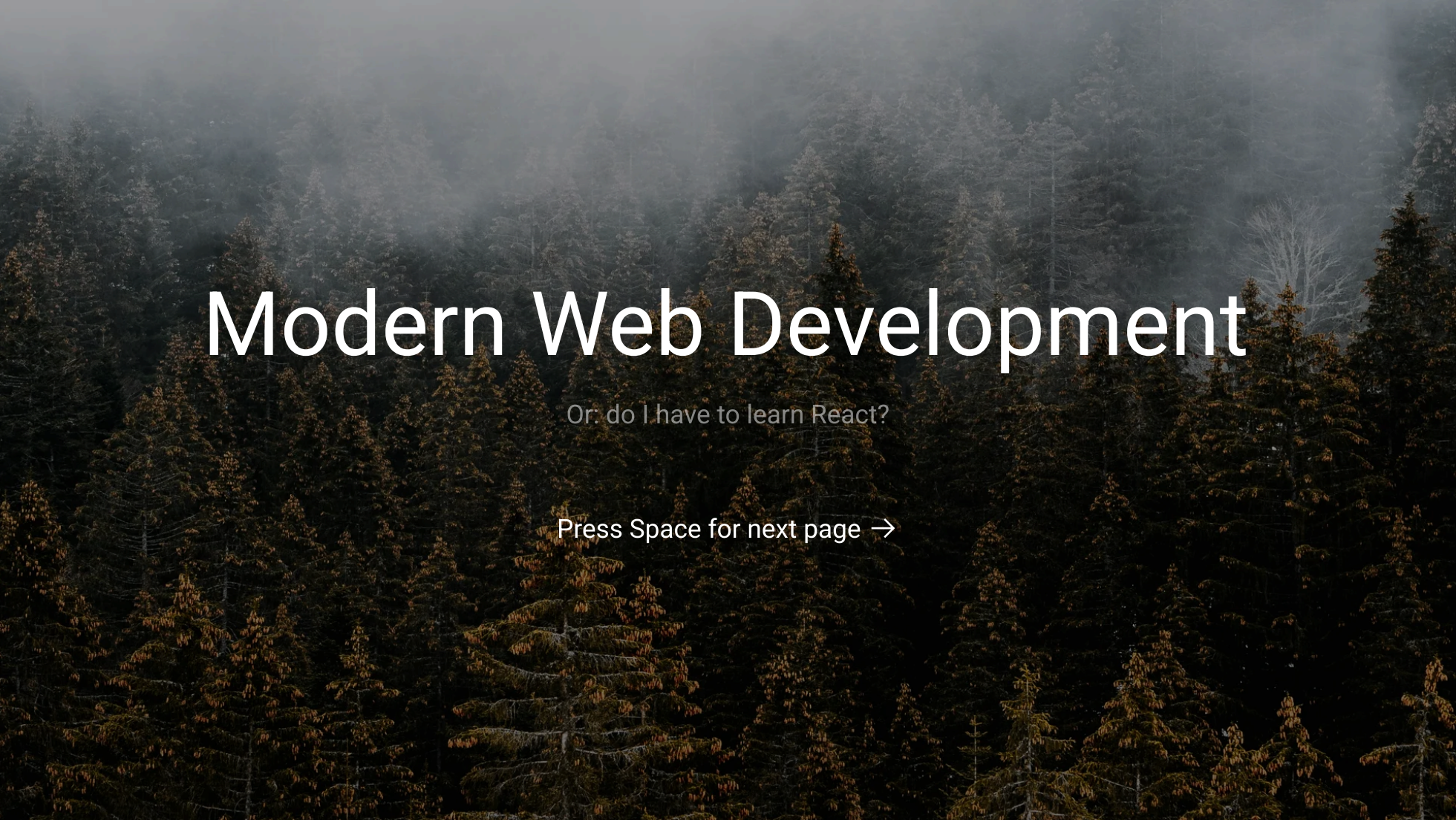


Modern Web Development

Or: do I have to learn React?

Press Space for next page →

See the web version of these slides at: https://marius-mather.github.io/webdev-presentation-2026/ - some interactive HTML embedded.

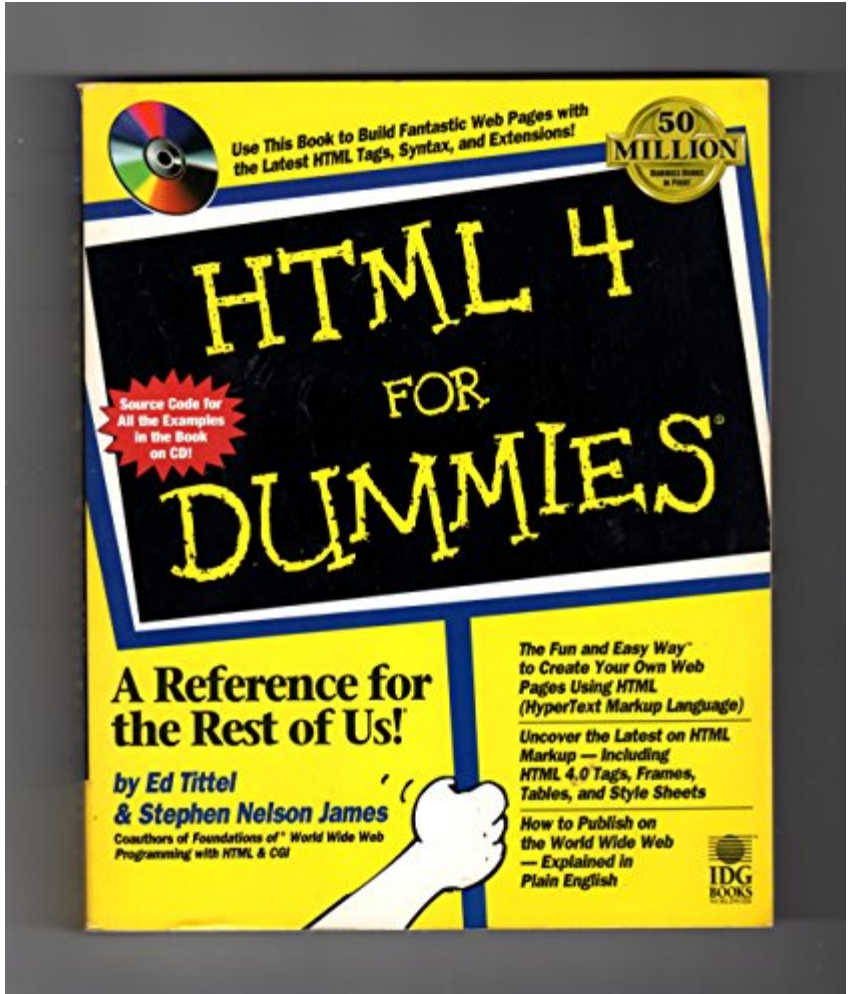
Isn’t it all just HTML?

Yes!

```
<h1>My web page!</h1>
<p>Welcome to my web site!</p>
```

My web page!

Welcome to my web site!


Use This Book to Build Fantastic Web Pages with the Latest HTML Tags, Syntax, and Extensions!
50 MILLION
HTML 4 FOR DUMMIES
Source Code for All the Examples in the Book on CD!
A Reference for the Rest of Us!
by Ed Tittel & Stephen Nelson James
Coauthors of Foundations of World Wide Web Programming with HTML & CGI
The Fun and Easy Way to Create Your Own Web Pages Using HTML (HyperText Markup Language)
Uncover the Latest on HTML Markup — Including HTML 4.0 Tags, Frames, Tables, and Style Sheets
How to Publish on the World Wide Web — Explained in Plain English
IDG BOOKS

It’s all still HTML + CSS + JavaScript

CSS

```
<style>
  .my-title {
    color: #F00;
    background-color: #00F;
  }
</style>
<h1 class="my-title">My web page!</h1>
<p>Welcome to my website!</p>
```

JavaScript

```
<h1 class="my-title">My web page!</h1>
<button id="counter" onClick="addCount">0</button>
<script>
  let count = 0;
  let counter = document.getElementById("counter");
  function addCount() {
    count += 1;
    document.getElementById("counter").innerText = count;
  }
  counter.addEventListener("click", addCount)
</script>
```

My web page!

0

Servers: the other part of the equation

You need a server for:

- Storing user data (especially private user data)
- Authentication/Login
- Dynamically generating pages
- Keeping track of info across pages (and across machines)

(some of this is less true in the modern web: OIDC tokens, LocalStorage, etc.)

Server templates: creating dynamic HTML

- Used in PHP, Django etc.
- Allows (some) programmatic reuse of your HTML - less repetition
- Server injects variables into HTML templates and sends the built page as HTML

```
<h1>{{ question.question_text }}</h1>
<ul>
{% for choice in question.choice_set.all %}
    <li>{{ choice.choice_text }}</li>
{% endfor %}
</ul>
```

From: https://docs.djangoproject.com/en/6.0/intro/tutorial03/

What’s wrong with writing HTML/JS/CSS?

- Maybe nothing? It still works fine

Presentation over??

- JavaScript APIs are clunky
- Doesn’t scale well as sites get bigger - lots of repetition, deeply nested elements, keeping lots of HTML + JavaScript + CSS files aligned
- Slow and inefficent to update the page interactively - basically doing live edits to the HTML
- Mobile meant you had to get your site working on two very different screen sizes

Modern web development

- JavaScript frameworks like React: assembling sites from self-contained **components**
- CSS frameworks like Tailwind: allow you to keep your styling close to the content
- TypeScript: add some structure to JavaScript's messiness
- Lots of libraries to achieve common tasks
- Bundlers and build tools: take all the output from React/Tailwind/TypeScript. and create HTML/CSS/JS

Modern web architecture (in very broad strokes)

- Frontend: uses a framework like React to populate your pages with data, make calls to the backend, dynamically update
- Backend: an API that sends and receives JSON data, e.g. FastAPI

React

Core idea: JSX components. Self-contained, reusable elements that manage their own state/data.

```
function MyButton(props: {label: string}) {
  const {label} = props;
  return (
    <button className="bg-blue-400">{label}</button>
  );
}

export default function MyApp() {
  return (
    <div>
      <h1>Welcome to my app</h1>
      <MyButton label="Click here" />
    </div>
  );
```

from: https://react.dev/learn

React components

Can contain:

- Plain HTML*
- Other components
- Children - can allow any other elements to appear inside the current component
- JavaScript logic
- Conditional rendering

React components: state

- **State**: components automatically update as state changes

```
function ScoreDisplay(props: {score: number}) {
  return <p>Your score: {score}</p>
}

function MyGame() {
  const [score, setScore] = useState(0);

  return (
    <>
    <button onClick={() => setScore(score + 1)} />
    <ScoreDisplay score={score} />
    </>
  )
}
```

Tailwind

- A whole bunch of small "utility classes" that represent individual styles
- Tailwind scans your files and only includes the ones you've used in the final CSS
- Built-in tools for responsive design e.g. `md:`, `lg:` - only apply this style on a screen that's medium or larger

```
<button class="w-20 bg-blue-500 text-white p-2 md:p-4">
  Click here!
</button>
```

Build tools/Bundlers/Libraries/Frameworks

- React itself won't create a site for you - you probably need a framework like NextJS
- Every framework comes with its own bundler/compiler/build tools
  - Hopefully these "just work"
- Install extra packages from npm
- Run/build your site:

```
npm run dev
```

A note on frontend apps

> ⚠ Warning
>
> Don’t trust the frontend!

- Can do all kinds of complex validation, checking, warning in the frontend (great for users!)
- API requests can always be sent separately from all of this
- Backend always has to check security, validity, logic.

What’s good about React

- Components are a good model
- Reactive updates are really cool
- Define your layouts once and reuse them
- Lots of great tools: linters, testers, libraries for whatever you need to do

Problems with React

- Lots and lots of JavaScript code to send to the client
- Some very confusing stuff that exists to serve the needs of huge companies - you probably don’t need Server Components
- Doesn’t necessarily have separate pages - different parts of your site might be generated on-demand through JavaScript
  - Can be bad for searchability
  - Frameworks like NextJS can do static exports (with some tweaking)
- Build tools are fantastic when they work, very frustrating when they don’t
- Everything changes constantly

Alternatives

- Lots of frameworks that try to stay closer to "just HTML": htmx, Svelte
  - All perfectly functional
  - A lot less support/tooling because they’re used less?
- Astro?
  - Built around static pages by default
  - Uses components: can mix and match Astro components, React, Angular
  - Support writing pages in Markdown (MDX), when the content is most important